

Jira Board

Resources

1. For drag and drop accessibility, use aria-grabbed and aria-dropeffect. Have keyboard navigation (eg: space to select item, ctrl+M to move items selected to current item) and the items must be focusable

Expectations

1. Components for rows and columns
2. APIs to get the required board data
3. Data model & Store

Functional requirements

1. Drag and drop across columns
2. Dynamic configuration of columns & column creation?
3. Edit a ticket
4. Multi-user support
5. Quick filters

Optional

1. Filter creation and application
2. Switch board

Non-Functional requirements

1. Mobile responsive
2. Good performance (infinite scroll & virtualization)
3. Accessibility
4. Observability/tracking
5. Internationalization
6. High Quality (Tests)
7. Security

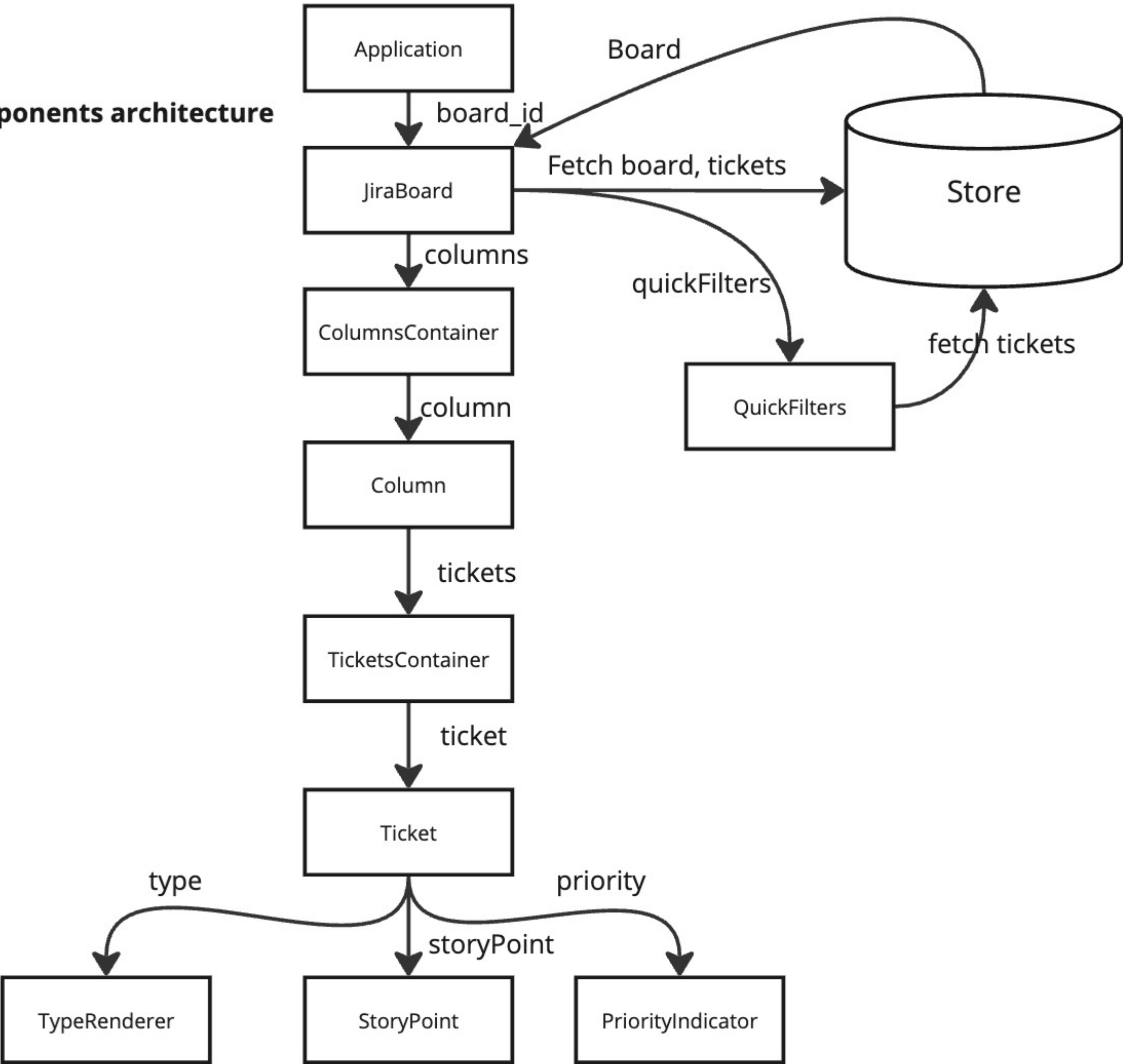
Questions

- 1. Design system available?
- 2. Mobile support
- 3. Should we support drag and drop?Support for concurrent users?
- 4. Are the columns predefined?Support to create a column
- 5. Should we support editing a card?
- 6. Quick filters?
- 7. Switch board?

Normalized store

```
board_store: { [board_id]: Board },
filters_store: { [board_id]: [Filter] },
tickets_store: {
  by_id: { [ticket_id]: JiraTicket },
  all_tickets: [ticket_id]
}
columns_store: { [board_id]: [BoardColumn] }
```

Components architecture



Data model

JiraBoard

```
{
  id: string;
  name: string;
  columns: [BoardColumn];
  quickFilters: [Filter];
  filter: Filter;
}
```

BoardColumn

```
{
  id: string;
  name: string;
  statusIds: [string];
}
```

Filter

```
{
  id: string;
  name: string;
  filterExpression: string;
}
```

JiraTicket

```
{
  id: string
  title: string
  description?: string
  storyPoints?: number
  type: JiraType
  assignee: JiraUser
  reporter: JiraUser
  priority: JiraPriority
  status: JiraStatus
}
```

enum JiraType{

```
  Bug
  Story
  Task
  Epic
}
```

enum JiraPriority {

```
  trivial
  minor
  normal
  major
  critical
  blocker
}
```

type JiraUser {

```
  id: string
  name: string
  email: string
}
```

type JiraStatus {

```
  id: string
  name: string
}
```


APIs

getBoard(boardId: string): JiraBoard

getTickets(page: cursorInput, filters: [Filter]): TicketsResponse

```
type cursorInput {  
  limit: Int  
  cursor: String  
}
```

```
type TicketsResponse {  
  tickets: [Ticket]!  
  pageInfo: PageInfo!  
}
```

```
type PageInfo {  
  prevCursor: String  
  nextCursor: String  
}
```

Additional talking points

1. Optimistic rendering
2. Infinite scroll and virtualization
3. Shimmers/loaders for better experience
4. Use SSR to render everything except the tickets. This can improve the time to first contentful paint

Performance

1. Network
 - a. Http2 -> Split modules and load in parallel - multiplexing
 - b. ES6/ES5 - separate packages so that we can load them based on browser support. They are loaded async and lazily. code splitting
 - c. Minification of JS & CSS
 - d. Gzip/brotli for compression
2. Rendering
 - a. Inline critical scripts & CSS
 - b. Defer loading non-critical scripts like analytics, telemetry etc
 - c. Use dns-preconnect and preconnect to connect to urls prior to making requests
3. Javascript
 - a. Use async operations so that the UI is not blocked
 - b. Use cache to cache the results with an expiry for offline content

Accessibility

1. Use HTML semantic elements if possible. Else use the equivalent roles (like grid)
2. Use aria-grabbed and aria-dropeffect for DD. Also use keyboard short-cuts
3. Support keyboard shortcuts for navigating across tickets, open detail view
4. Use rems instead of px to support different browser settings and also scaling wrt to zoom level
5. Use semantic HTML elements

Observability

All events and errors are tracked so that we can build custom reports and perform A/B testing. Error analysis also becomes possible

Testing

E2E tests using cypress, selenium or playwright
Component based testing for reusable components
Unit tests for pure JS functions
Run them on pipeline

Availability & Quality

1. Monitoring tool like new-relic on different regions
2. Pipeline runs all tests on various environments to make sure that no breaking changes get into high envs
3. ESLint to define development practices
4. Sonarcube for code smells, bugs, security vulnerability

Security

1. Use CSP headers to restrict the origin of scripts and resources to prevent XSS attacks
2. Use HTTP-only, secure and samesite cookies
3. Sanitize any user input to avoid XSS attacks

Real-time support

Check if absolute real-time support is required. Explain short, long polling, websockets and SSE.

Check the exact use case and choose one - mostly SSE

Internationalization

1. Localization - translation using react-intl
2. Localise date, number, currency - use new Intl.DateTimeFormat(locale).format(date), NumberFormat, toLocaleString(), toLocaleTimeString(), toLocaleDateString() etc
3. Localise urls
4. Handle LTR and RTL -
 - a. margin-left becomes margin-inline-start
 - b. margin-right becomes margin-inline-end
 - c. padding-left becomes padding-inline-start
 - d. padding-right becomes padding-inline-end
5. Use UTF-8 encoding
6. Handle accessibility
 - a. Use semantic html
 - b. Use alt
 - c. Ensure keyboard navigation
 - d. Have sufficient contrast of text with background
 - e. Use aria roles
 - f. User testing

UI Library

Agenda

- 1. Need
- 2. Organize - atomic design - bottom up - audit
- 3. How are the components organized
- 4. Framework
- 5. Package management
- 6. Bundler
- 7. Documentation
- 8. Testing
- 9. Quality & optimization
- 10. Accessibility
- 11. Support

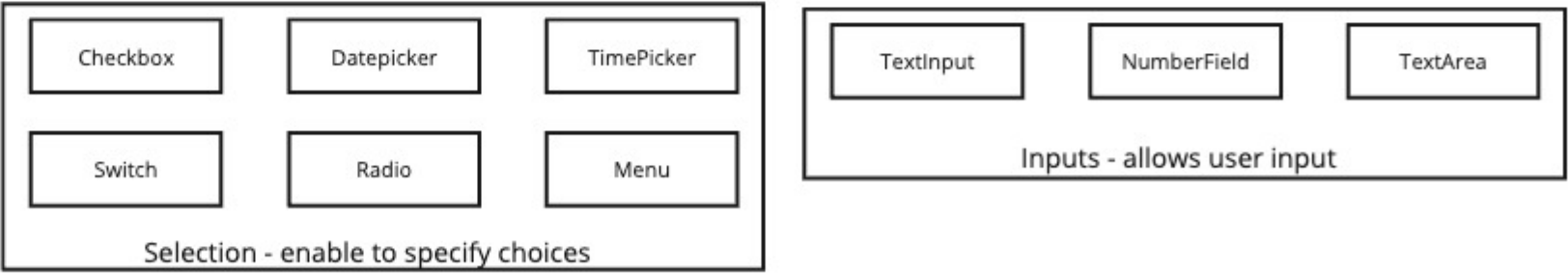
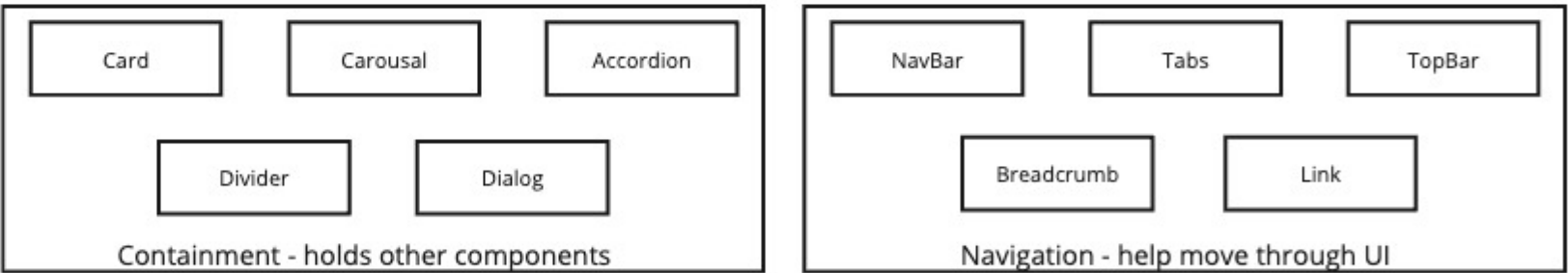
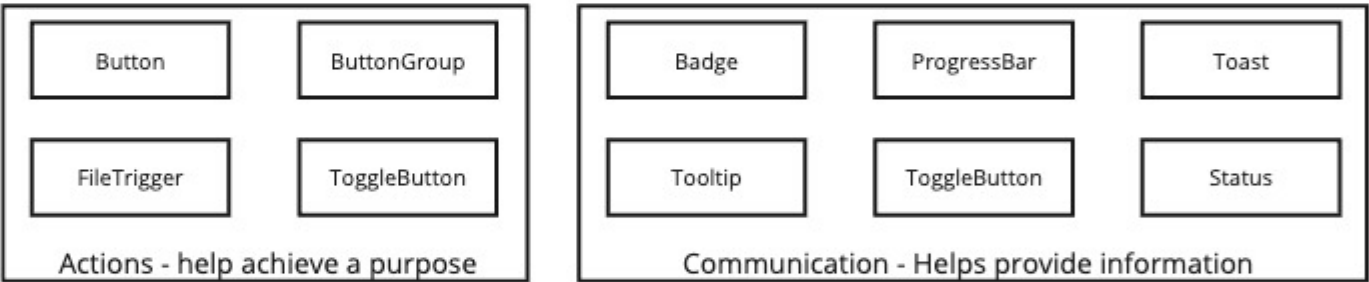
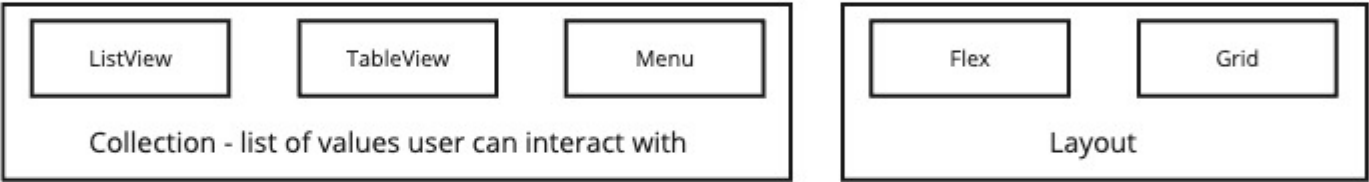
Introduction

- 1. A design library greatly **improves user experience** giving a **consistent look and feel** to the application.
- 2. Helps improve **developer velocity and productivity**
- 3. Avoids code duplication and **duplicate components** serving the same purpose

Audit the UI

- 1. Perform an UI audit as a first step to uncover the different components and patterns that are required for the application and also the present inconsistencies
- 2. Show-case the current inconsistencies in the components to justify the need for a UI components library
- 3. Then sort/organize the components into categories, which form the foundation of your library. Atomic design is one way of doing it.

- Atoms - Labels, Input fields, Buttons, Color palettes, Fonts, Icons, Animations, Design tokens
- Molecules - group of items - forms, tabs, accordions, label + search input + button is a search control
- Organisms - complex UI elements formed from molecules and atoms - header with logo, search etc, listbox, table, grid etc
- Templates - template for homepage, shopping, news feed etc - includes organisms and molecules
- Pages - templates in action



Principles

1. **Single responsibility principle (SRP):** Each component should have a single responsibility. Avoid creating monolithic components that handle too many tasks. This principle promotes **reusability** and makes components **easier to understand** and **maintain**
2. **Separation of concerns:** We should consider separate concerns such as logic, styling, and presentation

Repository

1. Monolithic - different teams work on the same code base - multiple parts of the app in a single repo.
 - a. Advantages
 - i. Single repository for all applications
 - ii. Easy to develop. All code is in 1 place
 - iii. All test in 1 application
 - iv. Great for small application - easy & fast to dev
 - v. Easy to deploy - 1 app - 1 CI/CD pipeline
 - vi. Easy to debug - only 1 place to look at
 - b. Disadvantages
 - i. Gets bloated over time
 - ii. Lots of features to be tested. Small change requires release of entire app release
 - iii. Restrictions on the tech used for different parts of the application
2. Microfrontends - Monolith broken into individual components with single responsibility/business functionality. Own infra and db. Release separately from others.
 - a. Advantages
 - i. Independent deployments - easy testing and deployment
 - ii. Own tech stack
 - iii. Scalability
 - iv. Localised impact when a bug is introduced
 - v. Easy maintenance
 - b. Disadvantages
 - i. If highly dependent - local running is difficult
 - ii. Problem on production becomes difficult to debug. Good monitoring can solve it to some extent
 - iii. Integration of services becomes difficult
 - iv. Code duplication
3. Monorepo - 1 repo with multiple components/projects. Individual projects can be deployed separately. Projects can reference other project's code. Lerna/pnpm/yarn workspaces/rush to run commands in multiple projects at once.
 - a. Advantages
 - i. Code/library reuse - no duplication
 - ii. With a monorepo, all projects use the same version of dependencies, eliminating "dependency hell."
 - iii. Changes across multiple projects can be committed atomically, maintaining consistency and making rollback easier if necessary.
 - iv. Individual projects can be built separately
 - b. Disadvantages
 - i. Over time, monorepos can become large, leading to longer clone times and increased storage usage.
 - ii. Without proper tooling and configurations, build times may increase as the size of the monorepo grows.
 - iii. Traditional version control systems may struggle with very large repositories.

Choose a framework

Framework agnostic - adoption becomes difficult, browser support is another thing to be checked.

React, Angular, Vue

Choose a framework that the company widely uses: **choose react**

Why mono repo

A multi repo approach makes it tough to apply any standardization across repos (like use typescript). There can be too many repos to handle and they become difficult to find. You will have to do too many npm links, which can get complicated

We go for mono repo because

- Dependencies do not break other packages since all packages are built and tested
- Easier to keep workflows and processes consistent across projects
- Reduces time spent pointing teammates in the direction of different repos
- Versions are synced across packages on master & your code always stays in sync.
- Single lint, build, test, and release process
- All documentation is in the same place
- Don't need to use yarn link when developing components that rely on each other
- You can make a single atomic commit that affects multiple packages and preserves that history and is easier to rollback (fixed)

Package management

1. **Yarn workspaces** does package management (installs once) and symlinks them together. It is efficient than lerna
2. **Lerna** also supports package management but can be used more for versioning and publishing (needs .npmrc with auth key)
3. Yarn workspaces and lerna can work together and is a great combination that compliments each other.
4. Rush has a centralized dependency system, where packages use the same version of dependencies. Lerna supports decentralized system and common versions are hoisted at the top.

Verdict: choose lerna + yarn workspaces

Bundler

1. Generates the ESM, CJS, UMD and AMD bundles for the code. Support tree shaking, code splitting etc
2. Rollup: Limited support for Hot Module Replacement, smaller community, dynamic imports are not seamless
3. Webpack: Large community, good hmr. But complex to setup, low performance in dev mode
4. Parcel: zero-config, limited configuration options, smaller community

Verdict: Webpack

Lerna - for reference

1. Command runner for a monorepo. Simplifies running commands for all packages in the monorepo.
2. Can also npm publish packages - there are a lot of options - auto inc version of all changed package or manual. Then publish all packages. Can also publish packages whose version is different from version of the prev published package (from-package). Else auto increments versions and publishes them.
3. Supports dependencies of job. For example, build of package1 should be done after build of package2
4. Supports concurrent execution
5. Run commands for packages changed after a given ref
6. lerna bootstrap does npm install for every package and links them
7. lerna exec yarn add <package> adds the package to all packages. You can define the package using a --scope too.

Documentation & storybooks

1. Allows build and testing in isolation
2. Collaboration with teammates and stakeholders
3. Helps find any component
4. Autodoc addon for documentation - args, argTypes, parameters, mdx referencing a story etc
5. Reuse these stories for unit tests in testing using react-testing-library
6. Getting started guide, online courses, boiler-plates (storybook)
7. Use for image (VR) testing - chromatic + test runner
8. Accessibility testing - Test runner + axe-playwright
9. Snapshot testing
10. interaction testing
11. E2E with cypress

Testing framework - ensure correctness, maintainability, bugs early

Jest - large community, includes spy, assertion library <- **better option**

Chai for assertion, sinon for spy and mocha as a test framework

Functional & snapshot testing

React testing library - based on UI (user centric) - tests the component behaviour <- **better option**

Enzyme - component centric - tests the internal state of the component

Image based testing against benchmark images

Percy - many false positives

Chromatic - integrates with storybook, great to test components <- **better option**

Mention about integration and E2E tests

Quality & optimization

1. Versioning - patch/bugfix, minor, major, alpha, beta. Codemods to auto upgrade codebase based on AST (abstract syntax tree).
2. Use virtualization and support infinite loading for collections
3. Measure TTI (time to interactive), TTR (time to render)
4. React profiler can be integrated in CI/CD to measure performance. Alerts when performance is below benchmark and we can block merging.
5. Browser compatibility must be assessed and polyfills included for graceful degradation
6. Typescript - Static typing at compile time - catch errors early - more predictable - self documentation
7. ESLint - JS is error prone - allows to discover problems without code execution - allows custom rules - enforces best practices - code consistency - eg: fragments instead of unnecessary containers - prop types not defined - correct use of hooks etc. Event accessibility rules can be used
8. Prettier to take care of formatting rules
9. Use css modules to avoid global namespace collision and have localized css for every component
10. SSR renderability - do not have window. or document. etc
11. Use useMemo, useCallback and memo wherever required
12. proper versioning invalidates the cache - webpack appends a hash to the files based on content
13. code splitting via dynamic imports
14. Good naming conventions - mirror dom when possible
15. Changelog and versioning history
16. Establish coding guidelines - PRs, Documents, contributing guide
17. Documentation of components - code documentation and prototypes
18. Use svg sprite generator to combine all sprites into 1 and use those by ids in the html code
19. es6 modules - less size due to effective tree shaking, supports lazy loading through dynamic imports (resulting in code splitting), better caching since individual modules are cached, not transpilation required

Observability

Monitor component usage is monitored so that we know which components are used and by which teams

Accessibility

- 1. Lighthouse for accessibility testing - also support it in build pipeline
- 2. Screenreader test with software like NVDA and others
- 3. Make sure that the appropriate roles and aria attrs are used
- 4. Correct semantic elements are used
- 5. Ensure keyboard navigation
- 6. Internationalization is handled
- 7. Support keyboard shortcuts for navigating across controls and buttons
- 8. Use rems instead of px to support different browser settings and also scaling wrt to zoom level

Support

- 1. Issues - JIRA
- 2. Github discussions
- 3. Jira project board to look at the plan for next quarters etc
- 4. Contributing guide
- 5. Versioning and changelog